Binary trees follow up

Student worksheet • Drawings, pointers and recursive in-order traversal
Name: _____ Date: _____

Work through Tasks 1–4 before coding. Ask for a check after Task 2 and Task 3. Task 5B and records are optional. These are original practice tasks, not official examination questions.

Task 1 Draw and explain

Work on paper. An ordered binary tree is also called a binary search tree (BST). Use smaller values on the left and larger values on the right. All values here are distinct.

A. Insert 40, 20, 60, 30 in that order. Draw a new sketch after each insertion. Label each left or right link L or R.

Sketch / working space

B. On your final sketch, circle the root, underline the leaves, and loop around the whole subtree rooted at 20.

C. Explain why 30 belongs to the left subtree of 40 even though it is a right child. Does every binary tree obey the BST ordering rule?

_____

_____

_____

Task 2 Link the picture to storage

Keep your Task 1 tree. Allocate rows in insertion order: 40 at index 0, 20 at 1, 60 at 2 and 30 at 3. A pointer stores a ROW INDEX. -1 means no child. Columns are 0 = left, 1 = data, 2 = right.

A. Fill every blank. RootPointer = _____. Both missing child pointers must be -1.

Index	Left (0)	Data (1)	Right (2)
0		40	
1		20	
2		60	
3		30	

B. Start at row 0 and follow its left pointer, then that row's right pointer. Write the indexes visited and the data values seen.

_____

_____

C. Explain the difference between TreeArray(1, 2) and TreeArray(TreeArray(1, 2), 1). Work out both values.

_____

_____

_____

D. A different tree has RootPointer = 2 and the table below. Draw it, with each node labelled index : data. Never assume the root is row 0.

Index	Left (0)	Data (1)	Right (2)
0	-1	9	-1
1	-1	4	-1
2	1	7	0

Sketch / working space

Task 3 Act out calls and returns

Use the SMALL tree below. RootPointer = 1. This is a binary tree but deliberately not a BST. Follow pointers rather than sorting the values. Draw the tree before tracing.

Index	Left (0)	Data (1)	Right (2)
0	-1	15	-1
1	2	25	0
2	-1	10	-1

Sketch / working space

A. Use the numbered rules below. Each call has its own p and place to resume. On a new call, put a paper frame on top of the waiting calls. On return, remove it and resume the caller.

```
1  If p = -1, return immediately.
2  Call InOrder(left pointer of row p).
3  Output the data in row p.
4  Call InOrder(right pointer of row p).
5  Return to the caller.
```

B. Complete the trace on the next page. The first two events are done. A return from -1 produces no output. Use indexes inside InOrder(...).

Task 3 continued

Event	Active p	What happens / where next?	Output added
1	1	Call InOrder(2); p=1 waits at rule 3	—
2	2	Call InOrder(-1); p=2 waits at rule 3	—
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			

C. Just before outputting the left leaf: what is the active p, what call is waiting underneath it, and at which rule will that waiting call resume?

_____

_____

D. Write the complete output. Explain why returning from the left subtree does not end the whole traversal.

_____

_____

_____

Task 4 Complete a recursive procedure

Return to the Task 1 tree and your Task 2A table. This version accepts -1 and checks it before reading the array. Complete the five gaps without looking at the full solution.

```
Sub InOrder(ByVal p As Integer)
    If p = (1)________ Then
        (2)________
    End If
    InOrder(TreeArray(p, (3)________))
    Console.WriteLine(TreeArray(p, (4)________))
    InOrder(TreeArray(p, (5)________))
End Sub
```

A. Explain what each of the five filled gaps does.

_____

_____

_____

_____

B. Which call starts the whole traversal? What happens if RootPointer = -1?

_____

_____

C. Predict the output for the Task 1 tree. Then draw the waiting calls when the procedure is about to output 30. Label each frame with p and its next action.

Sketch / working space

D. A student passes TreeArray(p, 1) to the recursive call. Explain the mistake and the likely effect.

_____

_____

Task 5 Repair and transfer

Use the Task 1 tree. Check the base case and the position of the output separately.

A. This procedure contains two independent mistakes. Identify each, explain its effect, and rewrite the corrected procedure below.

```
Sub Broken(ByVal p As Integer)
    Console.WriteLine(TreeArray(p, 1))  ' line A
    If p = -1 Then Exit Sub            ' line B
    Broken(TreeArray(p, 0))
    Broken(TreeArray(p, 2))
End Sub
```

Sketch / working space

B. Optional, after your teacher checks A: write PostOrder(p). It must accept a node index, safely handle -1, visit the left subtree, visit the right subtree, then output this node. Use only the 2D array.

Sketch / working space

C. Predict the post-order output for Task 1. Explain why the root appears last.

_____

_____

Task 6 Predict then run

Open Binary_Trees_Follow_Up_Starter.vb in a VB.NET console project. Save your version as BinaryTreePractice. The table is supplied so you can focus on recursion. Do not write an insertion routine.

A. Replace the TODO in InOrder with your completed Task 4 code. Before running, fill the prediction column. Then uncomment one test call at a time, run, and record the actual output. Write “no output” where appropriate.

Test call	Prediction	Actual output
InOrder(RootPointer)		
InOrder(1)		
InOrder(2)		
InOrder(-1)		

B. Put a breakpoint on the output line. Step through the main tree. At data 30, record p and the waiting calls. Explain where execution goes after the call for node 30 finishes.

_____

_____

_____

C. Change TreeArray(3, 1) from 30 to 35. Predict the new output before running. Why do the pointers stay the same? Restore 30 afterwards.

_____

_____

D. If ready: add your PostOrder procedure from Task 5 and test the whole tree, a leaf and -1. Record predictions first.

_____

_____

Exit check: A tree has root data 8 at index 2 and only a left child, data 5 at index 0. Draw it. Give the first call, the in-order output, and the point where the root call resumes after its left call.

Sketch / working space